

Week 05 Lab Instructions: OpenAPI 3.0 & Code Samples

Software Project Documentation · 15-Week Practical Course

National University of Mongolia
Department of Computer Science
Summer Semester 2026

Topic: OpenAPI 3.0, Swagger UI & Redoc

Timebox: 90 Min Lecture + 90 Min Lab

Instructor: Ph.D. Batnyam Battulga

Format: Practical Lab Exercise

SPRINT GOAL & STRATEGIC VISION

By the end of this lab, your guiding project repository must contain a valid **OpenAPI 3.0 specification (YAML)** covering at least **5 distinct endpoints** with complete authentication and component schemas. All embedded code samples must pass a strict quality audit based on **Bhatti's 5 Principles for Code Samples** (*Explained, Concise, Clear, Usable, Trustworthy*). The documentation must be live-rendered simultaneously via **Swagger UI** (Sandbox environment) and **Redoc** (Public reader view), accompanied by a 100-word comparative decision report.

PRODUCT BACKLOG (SPRINT USER STORIES)

TRACEABLE LITERATURE BASE

US-5.1 Machine-Readable OpenAPI 3.0 Specification

Chinchilla Ch. 2 / Bhatti Ch. 5

Lab Task 1

"As a Student Developer, I want to write an OpenAPI 3.0 YAML specification covering 5 endpoints with security schemas and reusable components, so that our API documentation is machine-readable and standardized."

Acceptance Criteria:

- YAML Validation:** File strictly validates against standard OpenAPI 3.0.3 using `@redocly/cli` without syntax errors or broken references.
- Complete Definitions:** All 5 endpoints define `path`, `parameters`, `requestBody`, and response codes (`200 OK`, `4xx`, `5xx`).
- Realistic Examples:** Every endpoint includes at least one `request-example` AND one `response-example` inside its payload schemas.

US-5.2 Bhatti 5-Principles Code Sample Quality Audit

Bhatti Ch. 5 (pp. 86–94)

Lab Task 2

"As a Student Technical Writer, I want to audit all code samples against Bhatti's 5 Principles, so that readers can copy-paste working requests without fixing broken placeholders."

Acceptance Criteria:

- Audit Scorecard:** Published audit table scoring every code sample from 1 to 5 stars across Explained, Concise, Clear, Usable, and Trustworthy metrics.
- Zero Generic Placeholders:** Replaced all `foo/bar/test` placeholders with real domain values (e.g., `your_pet_photo_url`).
- Trustworthy Payloads:** Responses match verified server payloads rather than invented mock structures.

US-5.3 & US-5.4 Dual Renderer Hosting & Comparative Selection Report

Chinchilla Ch. 4 & 6 / Bhatti Ch. 7

Lab Task 3 & 4

"As a Student DevOps Engineer, I want to host the OpenAPI spec on Swagger UI and Redoc simultaneously, so that our team can evaluate sandbox testing vs. public reading workflows."

Acceptance Criteria:

- Dual Endpoint Deployment:** Public URLs accessible for both Swagger UI (interactive sandbox) and Redoc (three-panel reference view).
- 100-Word Decision Report:** A concise trade-off summary uploaded to workspace evaluating search performance, layout readability, and "Try-It-Out" sandbox utility.

STEP-BY-STEP LAB EXECUTION GUIDE

EXECUTION TIME: 90 MINUTES

Step 1: Environment Setup & Linter Configuration

Terminal Setup

Initialize your workspace and install Redocly CLI to perform real-time linting during spec development:

```
# Install Redocly CLI globally or locally in your project
npm install -g @redocly/cli

# Create your spec file and run real-time linting
redocly lint docs/openapi/openapi.yaml --watch
```

Step 2: Authoring the OpenAPI 3.0 Spec Skeleton

YAML Authoring

Structure your `openapi.yaml` with security schemes, reusable component schemas, and 5 endpoints:

```
openapi: 3.0.3
info:
  title: Corgi.ly Pet Onboarding & Translation API
  version: 1.0.0
  description: Official API documentation for pet registration and voice translation.
servers:
  - url: https://api.corgi.ly/v1
    description: Production Server
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    PetProfile:
      type: object
      required: [pet_id, pet_name, breed]
      properties:
        pet_id: { type: string, example: "corgi_98231" }
        pet_name: { type: string, example: "Einstein" }
        photo_url: { type: string, example: "https://media.corgi.ly/photos/einstein.jpg" }
```

BHATTI’S 5 PRINCIPLES FOR QUALITY CODE SAMPLES

REFERENCE: BHATTI ET AL. (PP. 86–94)

Principle	Core Requirement & Book Guidance	Actionable Refactoring Requirement
1. Explained (p. 87)	Set context before the snippet. State what the code accomplishes and what input parameters are assumed.	Precede every code snippet with a 1–2 sentence explanation. Do not let standalone code float without prose context.
2. Concise (p. 90)	Strip unnecessary boilerplate (e.g., standard imports, heavy framework configurations). Focus purely on API logic.	Hide non-essential setup code. Highlight only the payload initialization and the client request call.
3. Clear (p. 92)	Use clean naming conventions, standard indentation, and explicit variable definitions. Avoid confusing shorthand.	Use descriptive variable names (e.g., <code>auth_token</code> instead of <code>t</code> ; <code>pet_response</code> instead of <code>res</code>).
4. Usable (p. 93)	Ensure copy-pasteability. Eliminate foo, bar, test, and meaningless acronyms.	Replace all <code>foo/bar</code> values with domain-specific placeholders (e.g., <code>your_pet_photo_url</code> or <code>replace_with_real_bark</code>).
5. Trustworthy (p. 94)	Ensure code is tested against live API responses. Never invent fake or outdated response structures.	Run the code sample against a real server/mock endpoint and embed the actual returned JSON response payload.

Step 4: Dual Renderer Hosting Setup

Deployment

Host Swagger UI and Redoc simultaneously in your repository workflow (e.g., via GitHub/GitLab Pages):

```
# Generate Redoc standalone HTML
npx @redocly/cli build-docs docs/openapi/openapi.yaml -o public/index.html

# Serve Swagger UI preview locally or via Docker
docker run -p 8080:8080 -e SWAGGER_JSON=/spec/openapi.yaml -v $(pwd)/docs/openapi:/spec swaggerapi/swagger-u
```

MANDATORY PRACTICAL TASK UE-5 (90 MIN)

SCENARIO: CORG.LY ONBOARDING API

UE-5

Corg.ly Onboarding API Code Sample Refactoring

Bhatti Ch. 5 (p. 83)

Mandatory Lab Task

Context: You are taking over Chad's legacy Corg.ly Onboarding API (Bhatti Ch. 5, p. 83). You must write 3 complete Python code samples for the endpoints below and audit them against Bhatti's 5 Principles:

- Endpoint 1:** POST /v1/pets/upload-photo (Pet photo upload with multipart data).
- Endpoint 2:** POST /v1/audio/translate-bark (Audio translation request returning bark meaning).
- Endpoint 3:** POST /v1/webhooks/subscribe (Registering a webhook URL for pet activity events).

Audit Scorecard Requirement (Complete in your submitted workspace):

Sample Code Target	Explained (1-5★)	Concise (1-5★)	Clear (1-5★)	Usable (1-5★)	Trustworthy (1-5★)
POST /v1/pets/upload-photo	★★★★★	★★★★☆	★★★	★★★★★ (No foo/bar)	★★★★★ (Live payload)
POST /v1/audio/translate-bark	★★★★☆	★★★★★	★★★	★★★★★ (Real audio path)	★★★★★ (Tested output)
POST /v1/webhooks/subscribe	★★★★★	★★★★★	★★★	★★★★★ (Domain hook URL)	★★★★★ (Status 201 verified)

Bonus (+10% Grade): Make at least one code sample fully executable and integrated into your repo's automated CI test pipeline (Bhatti p. 96 "Designing Code Samples").

AGILE CEREMONIES & QUALITY CHECKPOINTS

Daily Standup Prompts (3 Questions):

- How many valid endpoints are in our spec today? (Target: ≥ 5)
- Which code samples have we audited against Bhatti's rules?
- Which HTTP error response codes (4xx/5xx) are still missing?

Sprint Retrospective Questions:

- Which API authentication strategy did we choose and document?
- Where did generic placeholders (foo/bar) sneak into our draft?
- Why did our team choose Swagger UI vs. Redoc for public docs?

SPRINT 05 DEFINITION OF DONE (DOD) CHECKLIST

- ☐ OpenAPI 3.0 YAML spec committed to repository and passing redocly lint with zero errors.
- ☐ Public Swagger UI deployment accessible with functional "Try-It-Out" sandbox testing.
- ☐ Public Redoc deployment live with multi-column navigation and search functionality.
- ☐ Code Sample Audit Scorecard published with an overall average score of ≥ 80% (4.0/5.0 Stars).
- ☐ 100-word decision report comparing Swagger UI vs. Redoc uploaded to project workspace.

REQUIRED LITERATURE & REFLECTION PROMPTS

Required Readings	- Bhatti et al.: <i>Docs for Developers</i> (Apress 2021), Ch. 5 (pp. 83–99) & Ch. 7 (pp. 121–130). - Chinchilla: <i>Technical Writing for Software Developers</i> (Packt 2024), Ch. 2 (pp. 18–22) & Ch. 6 (p. 79).
Weekly Reflection Questions	- Which concept from Bhatti Ch. 5 challenged your previous coding/documentation habits the most? (Cite specific page). - Which concept from Chinchilla will you apply in future professional projects? (Cite specific page). - Where in your guiding project is the biggest gap between "what literature teaches" vs. "real implementation"?

